

LIMO ISA Specification

CS3520 — Computer Organisation and Architecture

Team: Architects

Repository: [saintchigos/limo-architects](https://github.com/saintchigos/limo-architects)

Milestone: M1

A1. Processor Style

LIMO is a 32-bit, load-store, RISC-V-derived instruction set architecture.

Property	LIMO Decision
Architecture class	RISC (load-store only)
Word size	32 bits
Memory model	Byte-addressed
Byte order	Little-endian
Instruction size	Fixed 32 bits
Instruction formats	R, I, S, B
Word alignment	Word-aligned (addresses must be multiples of 4)
Assembly language	Sesotho-language mnemonics

Load-store: only `ba1a` (load) and `boloka` (store) access data memory. All other instructions operate exclusively on registers.

Little-endian: for a 32-bit value stored at address A, the least significant byte is at A, the most significant byte at A+3. This matches RV32I and ensures straightforward cross-checking against Ripes.

Word-alignment: all instruction fetches and word memory accesses must use addresses divisible by 4. The simulator reports a misalignment error otherwise. This falls naturally from fixed 32-bit instructions: if the first instruction is at address 0, subsequent instructions are at 4, 8, 12, and so on.

Omitted formats: U-type and J-type are not included. LIMO's 12-bit I-type immediate is sufficient for all targeted programs. The absence of J-type means long-range unconditional

branches are not supported; this is documented as the primary structural limitation in A6(f). Any departure from RV32I field layouts is justified here.

A2. Register File

LIMO has **32 general-purpose registers** (r0–r31), each 32 bits wide. Register indices are 5 bits ($2^5 = 32$).

Programmer-visible special-purpose registers

Number	Assembly name	Sesotho meaning	Role
—	<code>sebali_tsamaiso</code>	program runner	Program Counter; not addressable as a numbered register
r0	<code>noto</code>	zero	Hardwired to 0; all writes are silently discarded
r1	<code>khutla_aterese</code>	return address	Stores return address on procedure calls
r2	<code>ts'upiso_phaello</code>	stack pointer	Points to the current top of the call stack
r3	<code>ts'upiso_kakaretso</code>	global pointer	Points to the global data region

General-purpose registers

Numbers	Group	Sesotho name	Role
r4–r7	Argument registers	<code>li_khang</code>	Function arguments / caller-saved
r8–r9	Return value registers	<code>li_phetho</code>	Function return values
r10–r15	Temporary registers	<code>li_nakoana</code>	Not preserved across calls
r16–r31	Saved registers	<code>li_boloka</code>	Callee-saved; preserved across calls

Note on `ts'upiso_phaello`: the apostrophe in `ts'` is a Sesotho digraph representing a distinct phoneme. See A4 for how the assembler handles apostrophes in identifiers.

A3. Instruction Set

LIMO implements **12 instructions** across four classes. No multiply/divide, floating-point, CSRs or exceptions are included.

Arithmetic — 3 instructions (including 1 immediate)

#	Mnemonic	Syntax	Operation	Format	RV32I equivalent
1	<code>kopanya</code>	<code>kopanya rd, rs1, rs2</code>	$rd = rs1 + rs2$	R	ADD
2	<code>tlosa</code>	<code>tlosa rd, rs1, rs2</code>	$rd = rs1 - rs2$	R	SUB
3	<code>kopanya_hang</code>	<code>kopanya_hang rd, rs1, imm</code>	$rd = rs1 + imm$	I	ADDI

Logic — 3 instructions (including 1 NOT)

#	Mnemonic	Syntax	Operation	Format	RV32I equivalent
4	<code>le</code>	<code>le rd, rs1, rs2</code>	$rd = rs1 \text{ AND } rs2$	R	AND
5	<code>kapa</code>	<code>kapa rd, rs1, rs2</code>	$rd = rs1 \text{ OR } rs2$	R	OR
6	<code>hase</code>	<code>hase rd, rs1</code>	$rd = \sim rs1$ (bitwise NOT)	R	— (pseudo: XORI rd, rs1, -1)

Note on `hase`: RISC-V has no dedicated NOT instruction; it uses `XORI rd, rs1, -1`. LIMO provides it as a real R-type instruction. The assembler sets $rs2 = r0$ (`noto`) and the processor ignores the $rs2$ field.

Shift — 1 instruction

#	Mnemonic	Syntax	Operation	Format	RV32I equivalent
7	<code>checha_hang_leqeleng</code>	<code>checha_hang_leqeleng rd, rs1, shamt</code>	$rd = rs1 \ll shamt$	I	SLLI

Only the lower 5 bits of the immediate ($shamt$) are used; a 32-bit register can only be shifted 0–31 positions.

Memory — 2 instructions

#	Mnemonic	Syntax	Operation	Format	RV32I equivalent
8	bala	bala rd, imm(rs1)	rd = Memory[rs1 + imm]	I	LW
9	boloka	boloka rs2, imm(rs1)	Memory[rs1 + imm] = rs2	S	SW

Branches — 3 instructions

#	Mnemonic	Syntax	Operation	Format	RV32I equivalent
10	lekana	lekana rs1, rs2, label	if rs1 == rs2: PC = label	B	BEQ
11	kholo_ho	kholo_ho rs1, rs2, label	if rs1 ≥ rs2: PC = label	B	BGE
12	nyane_ho	nyane_ho rs1, rs2, label	if rs1 ≤ rs2: PC = label	B	— (custom)

Note on nyane_ho: BLE is not in the RV32I base set. LIMO includes it as a custom branch using a unique funct3 value. The signed comparison is: branch taken if rs1 ≤ rs2. This departure is documented here per A1 policy.

A4. Sesotho Assembly

Mnemonic Glossary

Mnemonic	Sesotho meaning	Operation	RV32I equivalent
<code>kopanya</code>	combine / add together	$rd = rs1 + rs2$	ADD
<code>tlosa</code>	remove / take away	$rd = rs1 - rs2$	SUB
<code>kopanya_hang</code>	add right now / add immediately	$rd = rs1 + imm$	ADDI
<code>le</code>	and (conjunction)	$rd = rs1 \text{ AND } rs2$	AND
<code>kapa</code>	or (disjunction)	$rd = rs1 \text{ OR } rs2$	OR
<code>hase</code>	it is not / negate	$rd = \sim rs1$	—
<code>checha_hang_leqeleng</code>	shift immediately to the left	$rd = rs1 \ll shamt$	SLLI
<code>bala</code>	read / load	$rd = \text{Memory}[rs1 + imm]$	LW
<code>boloka</code>	save / store	$\text{Memory}[rs1 + imm] = rs2$	SW
<code>lekana</code>	equal / balanced	branch if $rs1 == rs2$	BEQ
<code>kholo_ho</code>	greater than or equal	branch if $rs1 \geq rs2$	BGE
<code>nyane_ho</code>	less than or equal	branch if $rs1 \leq rs2$	—

Assembly syntax

```
# This is a comment
label:
    mnemonic rd, rs1, rs2      # R-type
    mnemonic rd, rs1, imm     # I-type
    mnemonic rs2, imm(rs1)    # S-type
    mnemonic rs1, rs2, label  # B-type
```

Comments begin with (#). Labels end with (:). immediates may be decimal or hexadecimal (0x prefix).

Apostrophe handling

Sesotho orthography includes digraphs (ts') and (ch'), where the apostrophe denotes a distinct ejective consonant, not a delimiter. These appear in register names such as

(ts'upiso_phaello) and (ts'upiso_kakaretso).

Tokenisation rule: an identifier begins with a letter and continues consuming characters that are letters, digits, underscores (_), or apostrophes ('). Tokenisation stops at whitespace, a comma, a parenthesis, or end of line. This means (ts'upiso_phaello) is a single token.

Comment delimiter: the (#) character begins a comment, not the apostrophe. There is no ambiguity.

Example: (bala r1, 0(ts'upiso_phaello)) tokenises as: (bala), (r1), (,), (0), ((, ts'upiso_phaello),).

A5. Encoding and Specification

Instruction formats

All formats are identical in field layout to their RV32I counterparts. Field positions are preserved to maintain compatibility with RV32I tooling.

R-type

31	25 24	20 19	15 14	12 11	7 6	0
+-----+-----+-----+-----+-----+-----+						
funct7	rs2	rs1	fn3	rd	opcode	
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	
+-----+-----+-----+-----+-----+-----+						

I-type

31	20 19	15 14	12 11	7 6	0
+-----+-----+-----+-----+-----+					
imm[11:0]	rs1	fn3	rd	opcode	
12 bits	5 bits	3 bits	5 bits	7 bits	
+-----+-----+-----+-----+-----+					

S-type

```

31      25 24      20 19      15 14 12 11      7 6      0
+-----+-----+-----+-----+-----+-----+
|imm[11:5]| rs2  | rs1  | fn3  |imm[4:0]| opcode |
| 7 bits  | 5 bits| 5 bits| 3 bits| 5 bits | 7 bits |
+-----+-----+-----+-----+-----+-----+

```

B-type

```

31 30      25 24      20 19      15 14 12 11      8 7 6      0
+---+-----+-----+-----+-----+-----+---+-----+
|12|imm[10:5]| rs2  | rs1  | fn3  |imm[4:1]|11|opcode|
| | 6 bits  | 5 bits| 5 bits| 3 bits| 4 bits | |7 bits |
+---+-----+-----+-----+-----+-----+---+-----+

```

The B-type immediate encodes a 13-bit signed byte offset with bit 0 implicitly 0 (word-aligned targets). This gives a branch reach of ± 4 KB.

Opcode assignments

Format	Opcode (binary)	Opcode (hex)
R	1001101	0x4D
I	1001001	0x49
S	0101101	0x2D
B	1001110	0x4E

funct3 and funct7 assignments

R-type (opcode = 1001101)

Instruction	funct3	funct7
kopanya	000	0000000
tlosa	000	0000001
le	001	0000010
kapa	101	0000011
hase	010	0000100

I-type (opcode = 1001001)

Instruction	funct3
bala	000
kopanya_hang	100
checha_hang_leqeleng	101

S-type (opcode = 0101101)

Instruction	funct3
boloka	000

B-type (opcode = 1001110)

Instruction	funct3
kholo_ho	000
nyane_ho	001
lekana	010

Immediate ranges

Format	Immediate bits	Signed range
I	12-bit signed	−2048 to +2047
S	12-bit signed	−2048 to +2047
B	13-bit signed, bit 0 = 0	−4096 to +4094 (multiples of 2)

Hand-encoded examples (one per format)

R-type: `kopanya r3, r1, r2`

Field	funct7	rs2	rs1	funct3	rd	opcode
Name	—	r2	r1	ADD	r3	R-type
Binary	<code>0000000</code>	<code>00010</code>	<code>00001</code>	<code>000</code>	<code>00011</code>	<code>1001101</code>

Full 32-bit binary:

```
00000000 00010 00001 000 00011 1001101
= 00000000000010000001000000111001101
```

Hex: `0x002081CD`

I-type: `kopanya_hang r5, r1, 10`

Field	imm[11:0]	rs1	funct3	rd	opcode
Name	10	r1	ADDI	r5	I-type
Binary	<code>000000001010</code>	<code>00001</code>	<code>100</code>	<code>00101</code>	<code>1001001</code>

Full 32-bit binary:

```
0000000001010 00001 100 00101 1001001
= 0000000001010000001100001011001001
```

Hex: `0x00A0C2C9`

S-type: `boloka r3, 8(r2)`

Offset 8 = `000000001000` (12-bit). `imm[11:5] = 0000000`, `imm[4:0] = 01000`.

Field	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
Name	8[11:5]	r3	r2	SW	8[4:0]	S-type
Binary	<code>0000000</code>	<code>00011</code>	<code>00010</code>	<code>000</code>	<code>01000</code>	<code>0101101</code>

Full 32-bit binary:

```
00000000 00011 00010 000 01000 0101101
= 0000000000011000100000100000101101
```

Hex: `0x0031042D`

B-type: `lekana r1, r2, 8`

Offset +8 as 13-bit: `bit[12]=0`, `bit[11]=0`, `bits[10:5]=000000`, `bits[4:1]=0100`.

Field	imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode
Name	8[12]	8[10:5]	r2	r1	BEQ	8[4:1]	8[11]	B-type
Binary	<code>0</code>	<code>000000</code>	<code>00010</code>	<code>00001</code>	<code>010</code>	<code>0100</code>	<code>0</code>	<code>1001110</code>

Full 32-bit binary:

```
0 000000 00010 00001 010 0100 0 1001110
= 000000000010000001010010001001110
```

Hex: `0x0020A44E`

A6. Design-Decision Log

(Each answer is at most 120 words, with evidence from our own programs where specified.)

(a) Register-file size: cost and benefit

Choosing 32 registers means each register field is 5 bits wide. An R-type instruction spends 15 of its 32 bits on register addresses, leaving 17 bits for opcode and funct fields — identical to RV32I and sufficient for our 12-instruction set. The forwarding unit requires four 5-bit

comparators to detect RAW hazards across EX/MEM and MEM/WB stages. The benefit is low register pressure: in our sample programs, all intermediate values remain in registers throughout computation with no spills to data memory, keeping instruction counts low. The cost is those 15 encoding bits — a 16-register design would reclaim 3 bits, usable for wider immediates, but at the expense of increased register spilling in loops.

(b) Architectural vs microarchitectural registers

LIMO's architectural registers — the 32 GPRs, PC (`sebali_tsamaiso`), zero register (`noto`), stack pointer (`ts'upiso_phaello`) and return address (`khutla_aterese`) — are programmer-visible and fully specified in the ISA. The microarchitectural registers — IF/ID, ID/EX, EX/MEM and MEM/WB pipeline registers and the instruction register — exist only to carry data between pipeline stages and are invisible to software. The ISA must not specify them because they are an implementation detail: a valid LIMO implementation could use a different pipeline depth or out-of-order execution and still honour the ISA contract. Exposing pipeline registers in the ISA would permanently bind every implementation to one specific microarchitecture, eliminating the implementation freedom that separating ISA from microarchitecture exists to provide.

(c) Branch resolution stage and flush cost

LIMO resolves branches in the EX stage, where the ALU computes the comparison and the branch target is available. By EX, two instructions have already entered the pipeline behind the branch — one in IF and one in ID — so both must be flushed on a taken branch, costing 2 cycles per taken branch. Resolving in ID would reduce flushes to 1 but requires a dedicated comparator and branch-target adder in the decode stage, adding significant hardware complexity. Given LIMO's small instruction set and simulator implementation goals, EX resolution offers the better complexity tradeoff. Our sample program 3 (counting loop) shows taken branches are infrequent enough that the 2-cycle penalty has minimal overall CPI impact.

(d) Load-use stall under full forwarding

A load's result is not available until the end of the MEM stage. A dependent instruction needs that value at the start of its EX stage. Even with full forwarding, the value would need to travel backwards in time — from MEM of the load to EX of the dependent instruction in the same cycle. This is physically impossible; one stall cycle is mandatory.

	C1	C2	C3	C4	C5	C6
bala	IF	ID	EX	MEM	WB	
kopanya		IF	ID	***	EX	MEM WB

↑
 stall inserted here
 value exits MEM at end of C4
 kopanya needs it at start of EX
 forwarded from MEM/WB to EX in C5

The bubble (**) is inserted by the hazard unit, which stalls kopanya for one cycle until the load result can be forwarded from MEM/WB.

(e) Flags register: would LIMO benefit?

LIMO's branches (lekana), (kholo_ho), (nyane_ho) embed their comparison directly into the branch instruction, following the RISC-V model. A flags register would separate comparison from branching: a compare instruction sets flags, a branch reads them. This adds expressive flexibility but creates a new RAW hazard: every branch becomes dependent on the preceding compare, requiring the forwarding unit to track the flags register as additional architectural state. Since LIMO has no multiply or complex operations that naturally produce flag outputs, the added hazard complexity outweighs the benefit. Keeping comparisons inside branch instructions means one instruction, one pipeline pass, and no flag-forwarding logic — simpler hardware and no extra stall cases in the hazard unit.

(f) What breaks first as programs grow

With 32 registers, register pressure is the last concern — LIMO programs can hold many live values simultaneously without memory spills. Immediate range (12-bit signed, -2048 to +2047) limits large constants and array offsets but only affects specific access patterns.

Branch reach breaks first. LIMO's B-type immediate gives a signed 13-bit offset, reaching ±4 KB — roughly ±1000 instructions. As programs grow beyond this, forward and backward branches to distant labels exceed the encodable offset. RISC-V handles this with JAL for long-range jumps; LIMO has no J-type instruction, meaning large programs would require workarounds such as loading a target address into a register and branching indirectly. This is the first structural limitation a real LIMO program would encounter.

Sample Programs

All programs are written in LIMO assembly and may be assembled and run on the LIMO simulator. Expected register and memory states after execution are given for each program.

Program 1: Arithmetic and Logic

Computes $\text{result} = (a + b) \text{ AND } (a - b)$ where $a = 12, b = 5$.

asm

```
# LIM0 Program 1 - Arithmetic and Logic
# Computes: r5 = (r1 + r2) AND (r1 - r2)
# Expected results: r1=12, r2=5, r3=17, r4=7, r5=1

kopanya_hang r1, r0, 12      # r1 = 12          (a)
kopanya_hang r2, r0, 5       # r2 = 5          (b)
kopanya      r3, r1, r2      # r3 = 12 + 5 = 17 (sum)
tlosa        r4, r1, r2      # r4 = 12 - 5 = 7  (difference)
le           r5, r3, r4      # r5 = 17 AND 7 = 1 (bitwise AND)
#
# Verification:
# 17 = 10001
# 7  = 00111
# AND = 00001 = 1 ✓
```

Expected final register state:

Register	Value
r1	12
r2	5
r3	17
r4	7
r5	1

Instruction classes exercised: arithmetic (kopanya, tlosa, kopanya_hang), logic (le).

Program 2: Memory Load and Store

Loads a word from memory, doubles it by adding it to itself, and stores the result at the next word address.

asm

```

# LIM0 Program 2 – Memory Access
# Assumes Memory[100] = 21 is pre-loaded before execution
# Expected: Memory[100] = 21 (unchanged), Memory[104] = 42

kopanya_hang r1, r0, 100      # r1 = 100          (base address)
bala         r2, 0(r1)        # r2 = Memory[100] = 21
kopanya      r3, r2, r2       # r3 = 21 + 21 = 42  (double r2)
boloka       r3, 4(r1)        # Memory[104] = 42

```

Expected final register and memory state:

Register	Value
r1	100
r2	21
r3	42

Address	Value
100	21
104	42

Instruction classes exercised: arithmetic immediate (kopanya_hang), arithmetic (kopanya), memory load (bala), memory store (boloka).

Hazard present: `kopanya r3, r2, r2` immediately follows `bala r2, 0(r1)` — this is a load-use hazard. The simulator must insert one stall bubble before the `kopanya` instruction proceeds to EX. This program demonstrates A6(d) in practice.

Program 3: Conditional Loop (Accumulator)

Computes the sum $5 + 4 + 3 + 2 + 1 = 15$ using a countdown loop.

```
asm
```

```

# LIM0 Program 3 – Conditional Branch and Loop
# Computes: r5 = 5 + 4 + 3 + 2 + 1 = 15
# r1 = loop counter (5 down to 1)
# r2 = decrement (constant 1)
# r3 = loop exit threshold (constant 1)
# r5 = accumulator

kopanya_hang r1, r0, 5      # r1 = 5 (counter, starts at 5)
kopanya_hang r2, r0, 1      # r2 = 1 (decrement value)
kopanya_hang r3, r0, 1      # r3 = 1 (loop exit threshold)
kopanya_hang r5, r0, 0      # r5 = 0 (accumulator)

boelela:
    kopanya r5, r5, r1      # r5 = r5 + r1 (accumulate counter)
    tlosa   r1, r1, r2      # r1 = r1 - 1 (decrement counter)
    kholo_ho r1, r3, boelela # if r1 >= 1, branch back to boelela

# Loop terminates when r1 = 0, at which point r1 >= 1 is false
# r5 = 5 + 4 + 3 + 2 + 1 = 15

```

Expected final register state:

Register	Value	Notes
r1	0	counter exhausted
r2	1	unchanged
r3	1	unchanged
r5	15	= 5+4+3+2+1

Cycle trace (loop body only, iterations 1-5):

Iteration	r1 before	Added to r5	r1 after	Branch taken?
1	5	5	4	Yes
2	4	4	3	Yes
3	3	3	2	Yes
4	2	2	1	Yes
5	1	1	0	No (exit)

Instruction classes exercised: arithmetic immediate (kopanya_hang), arithmetic (kopanya, tlosa), conditional branch (kholo_ho). This program also exercises the branch flush mechanism: each taken branch in the loop body flushes 2 instructions from the pipeline (A6(c)), visible in the simulator's pipeline chart as bubbles on the cycle following each taken `kholo_ho`.

Label: `boelela` — Sesotho for "go back / repeat".

End of LIMO ISA Specification — M1